Chapter 14 worked solutions • Protection

OS14-E01 • Domains and least privilege

1. Static D={readS, writeT} lets A write T, B read S, and C do both. Use DA={readS}, DB={writeT}, DC=∅ with authorized transitions and necessary data transfer. Do not pass arbitrary memory/full-authority handles across boundaries.
2. No. CPU mode separates privilege classes, not every process/domain. Address-space mappings and permissions still matter. Functions within one user-mode process do not thereby acquire memory isolation. Domain transitions also require authorization.

OS14-E02 • Exact matrix updates

1. After limited copy, D1={r*, owner}, D2={w}, D3={r}. Under this question's transfer of the entire r* capability including read, the next column is D1={owner}, D2={w, r*}, D3={r}. A representation separating r and the copy flag needs separately specified transfer semantics; it cannot silently retain source r.
2. Owner(F) removes D2's w from F's column; control(D2) removes it from D2's row. Control cannot add x. Adding x as F's owner uses a different rule. Same-column copy cannot create G rights.

OS14-E03 • Reachable domains versus simultaneous rights

1. Direct print fails because A[D1, P]=∅. Switching D1→D2→D3 then printing is legal. Future authority depends on transition reachability.
2. Initially reachable set={D1}. Every allowed successor remains in {D1, D2}, because D2 has no edge to D3 and no other path exists. Induction over trace length therefore excludes D3. Both reachable domains lack print, proving denial. Re-adding edges through owner operations would violate the frozen, switch-only premise.

OS14-E04 • ACLs, capabilities, and existing opens

1. U1={F:r}; U2={F:rw, G:r}; U3={G:w}. Each nonempty matrix entry occurs once; changing index direction changes no permission.
2. Write remains denied because the handle lacks w. Read depends on revocation. Open-time ACL checking with a live handle may permit continued read. Per-use generation validation rejects a revoked generation. ACL deletion alone cannot determine all existing-handle behavior. Deny/order-based ACL systems require their own semantics.

Chapter 14 worked solutions • Protection (continued)

OS14-E05 • Generation numbers and ABA

1. Slot-only checking grants G to the old cap. An unwrapped successor table5=(G, g4, live) rejects g3≠g4. Check the generation and retain a live object under one synchronization boundary, or replacement can race after checking.
2. A two-bit counter follows3→0→1→2→3 in four reuses. With the old token alive this is ABA. Forbid slot reuse before wrap, or establish global quiescence proving all old tokens gone. A large finite counter delays wrap without formally eliminating it.

OS14-E06 • Active RBAC roles

1. No active role grants writeGrades, so deny. Disable Backup before activating Grade; validate the exclusion constraint and Grade∈assigned(U). Assignment is not activation.
2. No. The rule is session-scoped. To prevent one person holding both across sessions, specify a user-level or static separation constraint. The mechanism cannot supply an unstated policy scope. UA, PA, and session state differ.

OS14-E07 • Historical Java inspection

1. Ordinary N passes, L passes, A fails, so deny. At privileged L, N passes, L passes its own p check, then inspection stops and allows without checking A.
2. L lacking p fails even when marked privileged. If no privileged stop intervenes and all explicit frames pass, inherited context must still hold p; otherwise deny.
3. A borrows L's broader network scope to reach forbidden targets. Stack checks can be correct while service policy fails. Restrict actual address, port, and operation; unvalidated caller parameters are not authority. A privileged caller must itself possess p.

OS14-E08 • Delegation and information flow

1. Without copy, owner, or another grant rule, D1 cannot delegate F:r. It can still read F's bytes and write them to G. The copy right regulates authority propagation rather than data copying.
2. Induction assumes every new r has an already-held source r. Owner may add a valid right not previously held by any domain in its column, breaking that premise. Prove the owner's right-creation scope separately. Authorization safety and confidential information flow are distinct claims.

Chapter 14 worked solutions • Protection (continued)

Verification code appendix: access_matrix.py

```
"""Access matrix and revocation checks.

Run:
    python access_matrix.py
"""

Matrix = dict[str, dict[str, set[str]]]


def allowed(matrix: Matrix, domain: str, obj: str, right: str) -> bool:
    return right in matrix.get(domain, {}).get(obj, set())


def revoke(matrix: Matrix, domain: str, obj: str, right: str) -> None:
    rights = matrix.get(domain, {}).get(obj, set())
    rights.discard(right)


def main() -> None:
    matrix: Matrix = {
        "D1": {"F1": {"r", "w"}, "F2": set()},
        "D2": {"F1": {"r"}, "F2": {"w"}},
    }
    assert allowed(matrix, "D1", "F1", "w")
    revoke(matrix, "D1", "F1", "w")
    assert not allowed(matrix, "D1", "F1", "w")
    assert allowed(matrix, "D1", "F1", "r")
    assert allowed(matrix, "D2", "F2", "w")
    print(matrix)


if __name__ == "__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 14 worked solutions • Protection (continued)

Verification code appendix: protection_models.py

```
"""Finite teaching cases; authority safety proofs are in the chapter."""
def reachable(start,edges):
 seen={start};stack=[start]
 while stack:
  for target in edges.get(stack.pop(),()):
   if target not in seen:seen.add(target);stack.append(target)
 return seen
def resolve(cap,entry):
 slot,g,right=cap;obj,version,live,rights=entry
 return obj if live and g==version and right in rights else None
def inspect(frames,inherited):
 for permitted,privileged in frames:
  if not permitted:return False
  if privileged:return True
 return inherited
if __name__=='__main__':
 assert reachable('D1',{'D1':['D2'],'D2':['D3']})=={'D1','D2','D3'}
 assert reachable('D1',{'D1':['D2']})=={'D1','D2'}
 assert resolve((7,12,'r'),('F',12,True,{'r'}))=='F'
 assert resolve((7,12,'r'),('G',13,True,{'r'})) is None
 assert not inspect([(True,False),(True,False),(False,False)],True)
 assert inspect([(True,False),(True,True),(False,False)],True)
 assert not inspect([(True,False),(False,True),(False,False)],True)
 assert not inspect([(True,False)],False)
 print('switch reachability, versioned handle and four stack cases passed')
```

Lin Ching https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study